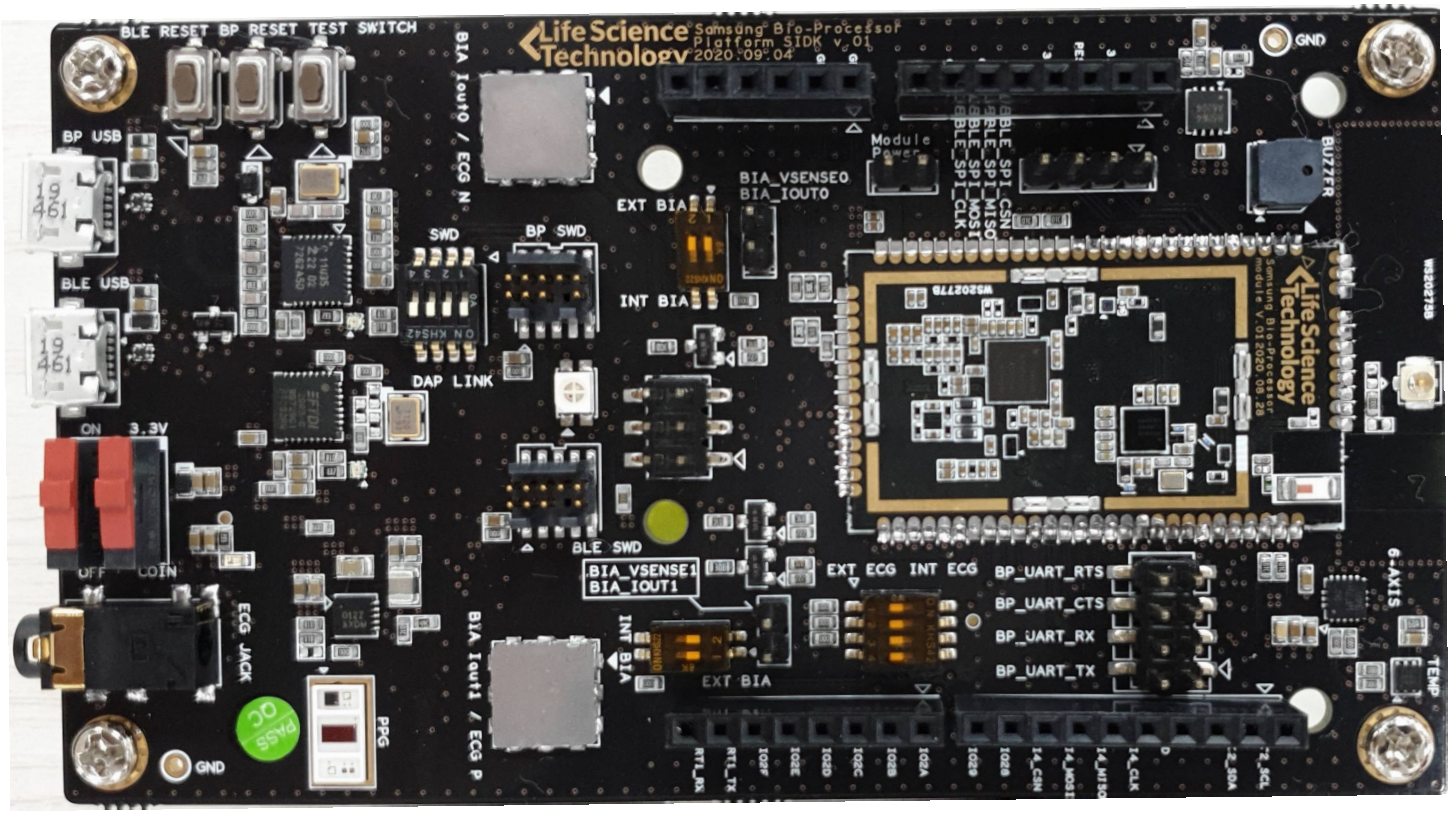
BioProcessor 시작하기



목차

1. BioProcessor
2. 환경 구성
3. 빌드 및 배포
4. 실행, 디버깅
5. API Reference

1. BioProcessor

1.1.BioProcessor

BioProcessor는 생체신호 연구 • 개발용 오픈 플랫폼입니다.

BioProcessor 개발 보드는 ECG, PPG, BIA 등 생체 신호를 측정하는 센서와 통신을 위한 BLE 모듈이 결합된 형태이기 때문에 생체 신호를 연구하고 헬스케어 제품을 개발하는 과정을 단축시킬 수 있습니다.

BioProcessor와 교육 프로그램을 이용하면 생체 신호를 이해하고 측정할 수 있으며 모바일 기술을 학습할 수 있습니다.

생체 신호를 측정하는 원리를 이해하고 계측할 수 있습니다. 또한 계측한 생체 신호를 통해 심박수, 체지방 등을 산출하는 기본적인 알고리즘을 학습할 수 있습니다.

BioProcessor의 BLE 모듈을 이용해 무선 통신 기술을 이해하고, 디지털 신호 처리를 이해할 수 있습니다.

1.2. ARM Mbed OS

BioProcessor는 오픈소스 임베디드 운영체제인 ARM Mbed OS를 이용해 플랫폼을 구축하고 있습니다.

ARM Mbed OS는 ARM Cortex-M 마이크로 컨트롤러를 사용한 제품을 개발하기 위한 오픈 소스 운영체제로, 보안, 연결, RTOS, 그리고 센서와 I/O장치의 드라이버를 모두 포함하는 운영체제입니다.

통합 개발 환경(IDE)를 제공하기 때문에 개발이 쉽습니다.

1.3. BioProcessor 시작하기

이 가이드는 BioProcessor용 예제 프로그램을 빌드하고 BioProcessor에 설치합니다.

Mbed OS를 위한 IDE인 Mbed Studio를 사용하거나 Mbed CLI를 사용해 CLI 환경에서 프로그램을 빌드하고 배포할 수 있습니다.

모든 과정은 Windows 기준입니다. macOS 또는 Linux 배포판을 사용할 때는 각 항목 상단에 있는 문서 링크를 참조하여 설치할 수 있습니다.

2. 환경 구성

개발 환경을 구성합니다. GUI를 지원하는 IDE인 Mbed Studio를 설치하거나, 개발 유연성이 좋은 Mbed CLI를 선택하여 설치합니다.

- **Mbed Studio**

  Mbed Studio는 Mbed OS 개발을 위해 특별히 제작된 IDE입니다.

  Mbed OS 개발에 필요한 의존성과 도구들이 모두 한 패키지에 포함되어 있습니다.

  BioProcessor가 연결되면 BioProcessor를 자동으로 인식하고 프로그램을 설치할 수 있습니다.

  제한적으로 무료로 배포되는 ARM Compiler 6를 완전히 무료로 제공합니다.

- **Mbed CLI**

  Mbed OS 개발에 필요한 도구를 수동으로 설치해야 하지만 최고의 개발 유연성을 제공합니다.

2.1. Mbed Studio

2.1.1. Mbed Studio 설치

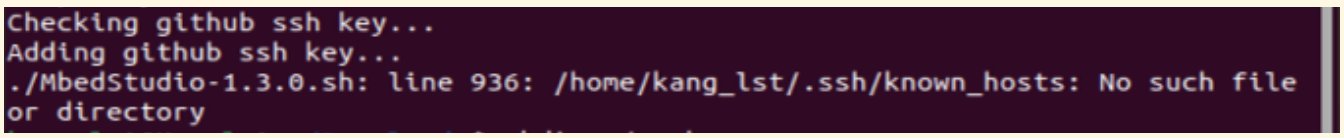
다른 운영체제는 아래 문서를 참조하시기 바랍니다.

https://os.mbed.com/docs/mbed-studio/current/installing/installing-mbed-studio.html#on-macos

https://os.mbed.com/docs/mbed-studio/current/installing/installing-mbed-studio.html#on-linux

Linux의 경우 .ssh 디렉터리가 없을 경우 오류가 발생합니다.

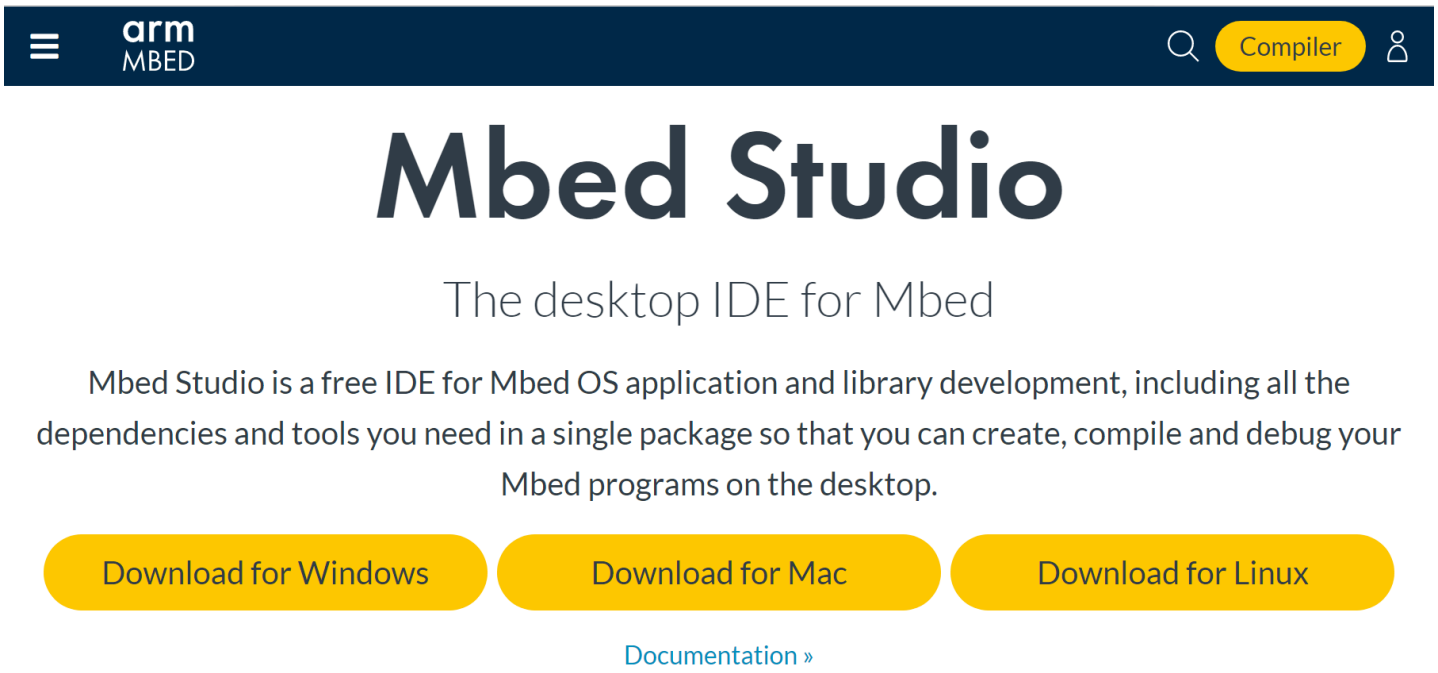
github에 SSH로 연결하기 위해 known_hosts에 등록하는데, 그 파일을 .ssh 디렉터리 안에 생성합니다.



위와 같은 오류가 발생한다면 .ssh 디렉터리를 생성해야 합니다.

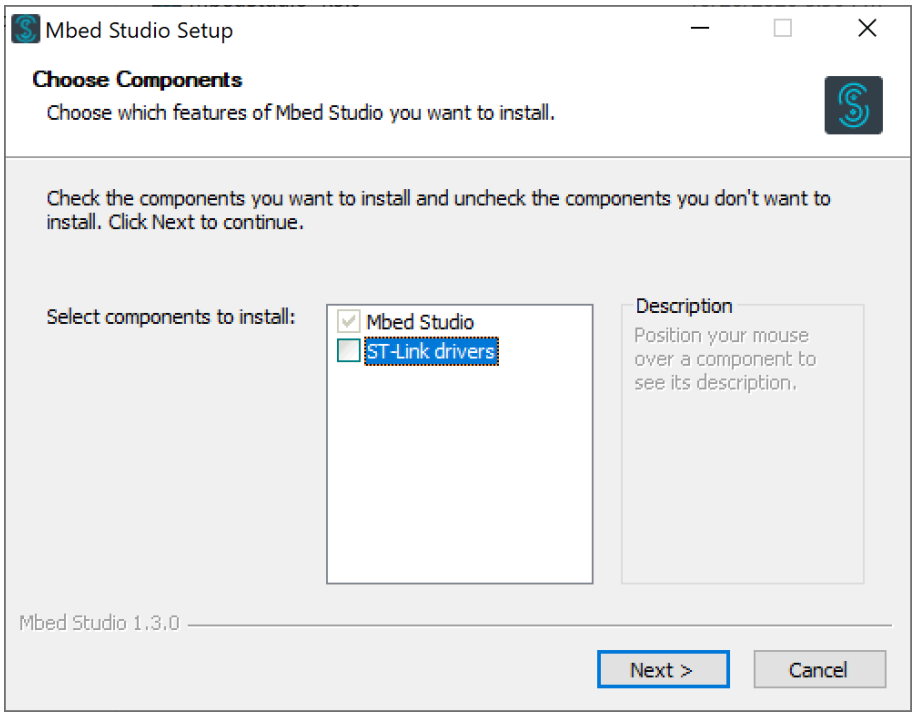
```
$ mkdir $HOME/.ssh
```

- https://os.mbed.com/studio/에서 OS에 맞는 IDE 다운로드



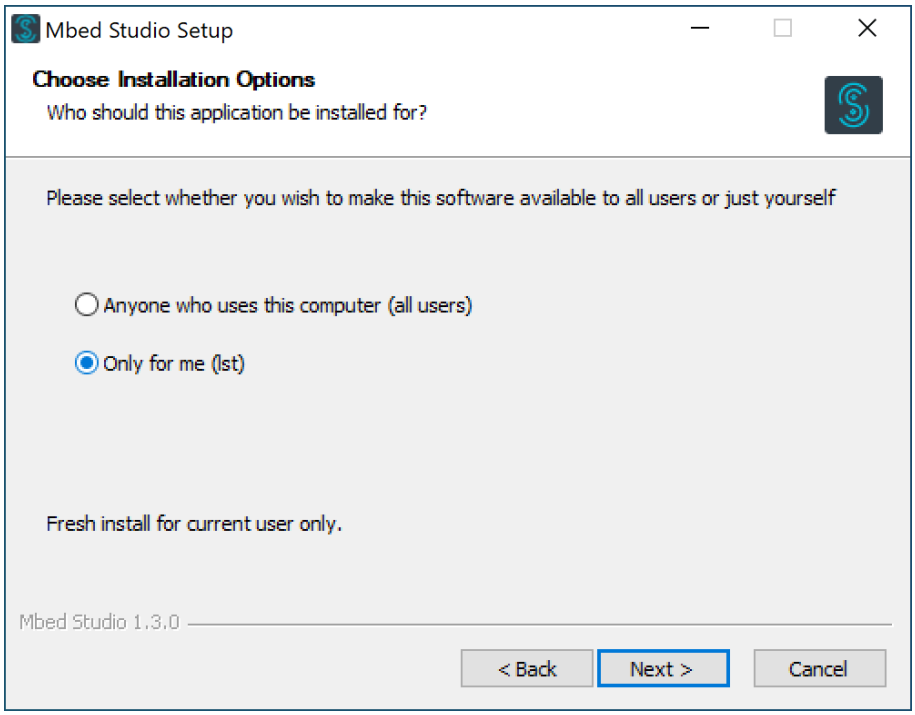
다운로드 페이지에서 운영체제를 선택하여 최신 Mbed Studio의 설치 프로그램을 다운로드 합니다.

- Mbed Studio 설치 프로그램 실행



Mbed Studio에서 BioProcessor를 사용하는데 ST-Link drivers는 필요하지 않습니다.

- Mbed Studio의 설치 범위 설정

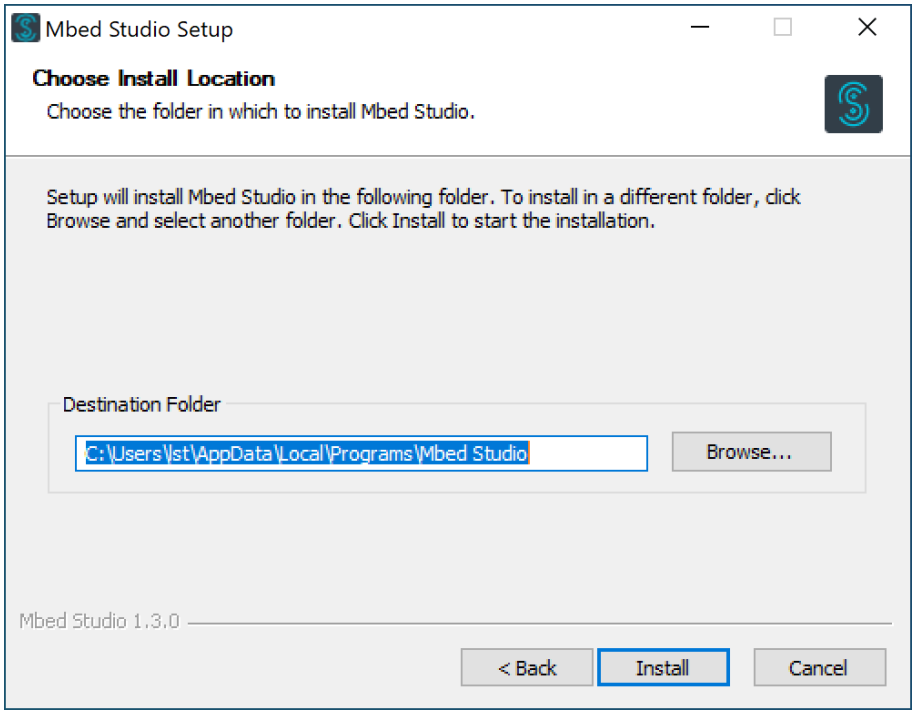


Mbed Studio를 실행할 수 있는 사용자를 지정합니다.

여러 사용자가 있는 컴퓨터에서 다른 계정에 영향을 주지 않으려면 Only for me를 선택합니다.

모든 사용자가 Mbed Studio를 실행할 수 있게 하려면 Anyone who uses this computer (all users)를 선택합니다.

- Mbed Studio의 설치 위치 설정

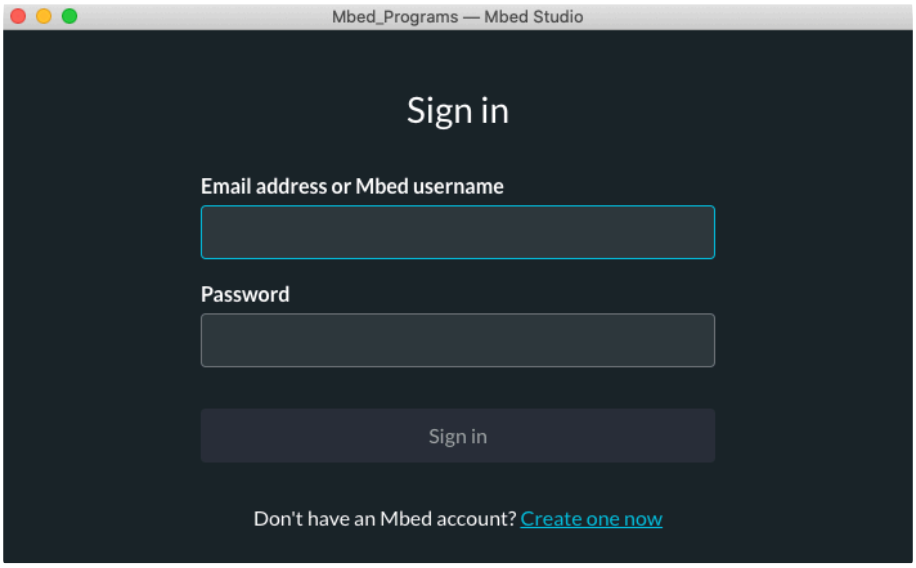


기본값으로 두고 Install 버튼을 클릭하여 설치를 시작합니다.

Mbed Studio의 설치가 끝났습니다. Mbed Studio를 실행하면 로그인 화면이 나옵니다.

2.1.2. Mbed Studio 회원 가입 및 로그인

Mbed Studio를 처음 실행하면 아래와 같은 로그인 화면이 나옵니다.



Mbed Studio를 사용하려면 mbed.com에 로그인해야 합니다.

계정이 없다면 ‘Create one now’ 버튼을 클릭하여 계정을 생성하고 로그인 할 수 있습니다.

이제 Mbed Studio를 사용하기 위한 환경 구성이 끝났습니다.

2.2. Mbed CLI

2.2.1. 필수 프로그램 (Python 3.7.x, Git, Mercurial) 설치

Mbed CLI는 Python 모듈이기 때문에 Python이 필요합니다.

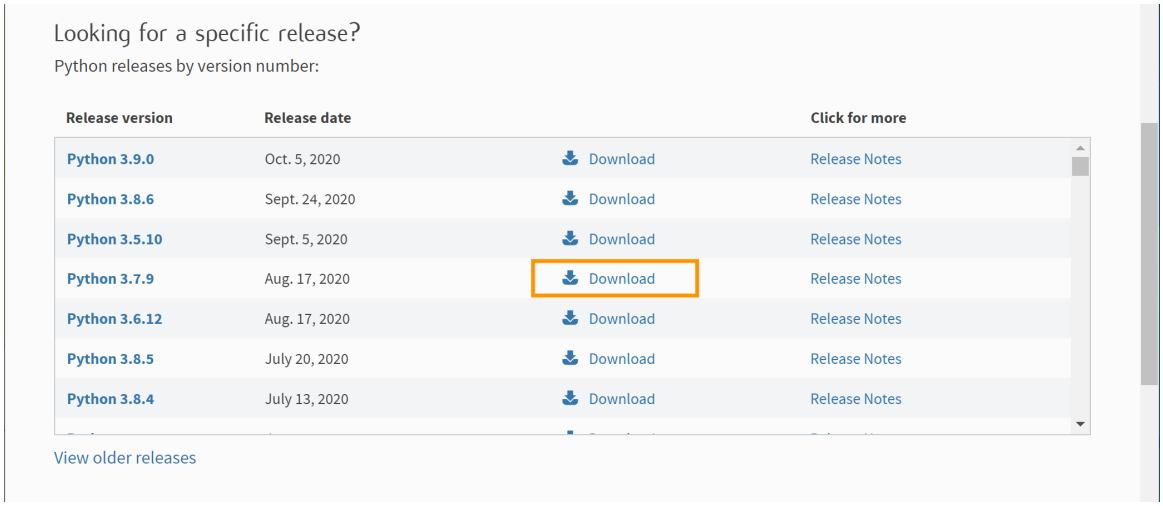
Mbed OS와 예제 프로그램을 다운로드 하는데 Git과 Mercurial이 필요합니다.

다른 운영체제는 아래 문서를 참조하시기 바랍니다.

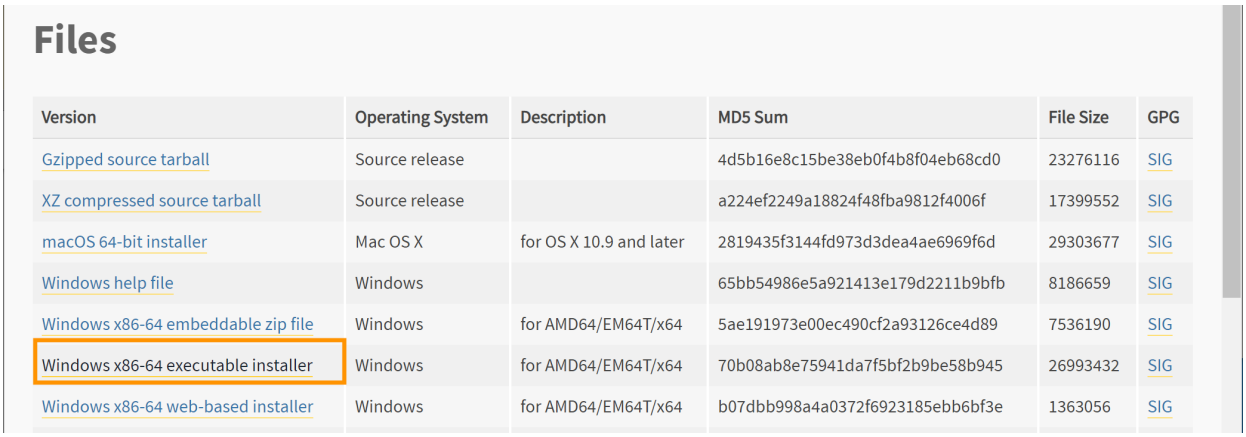
https://os.mbed.com/docs/mbed-os/v6.5/build-tools/install-and-set-up.html

- **Python 3.7.x 설치 : https://www.python.org/downloads/**
  - **다운로드 페이지 접속**

    Looking for a specific release? 부분에서 Python 3.7.9 버전을 다운로드합니다.

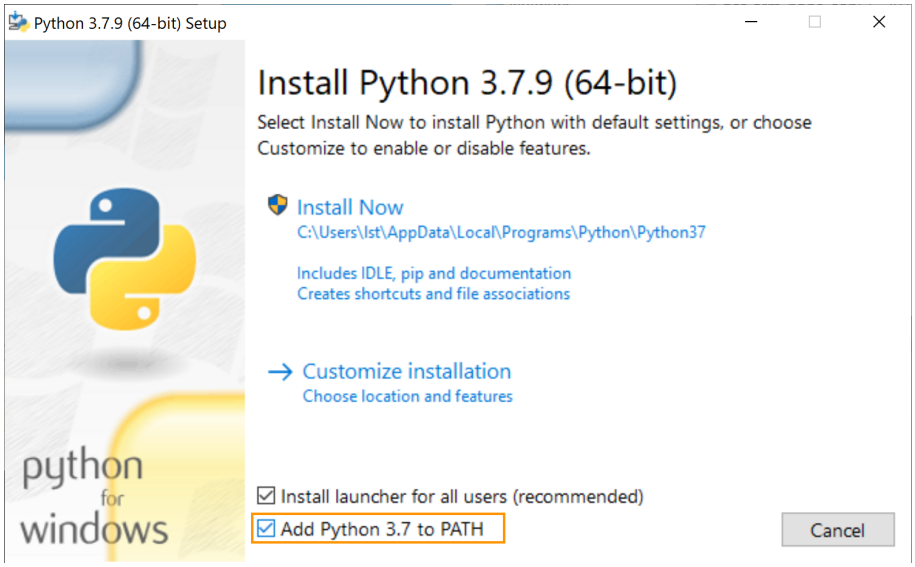


  - **Windows용 설치 프로그램 다운로드**



    Windows x86-64 executable install를 다운로드합니다.

- **Python 설치 프로그램 실행**

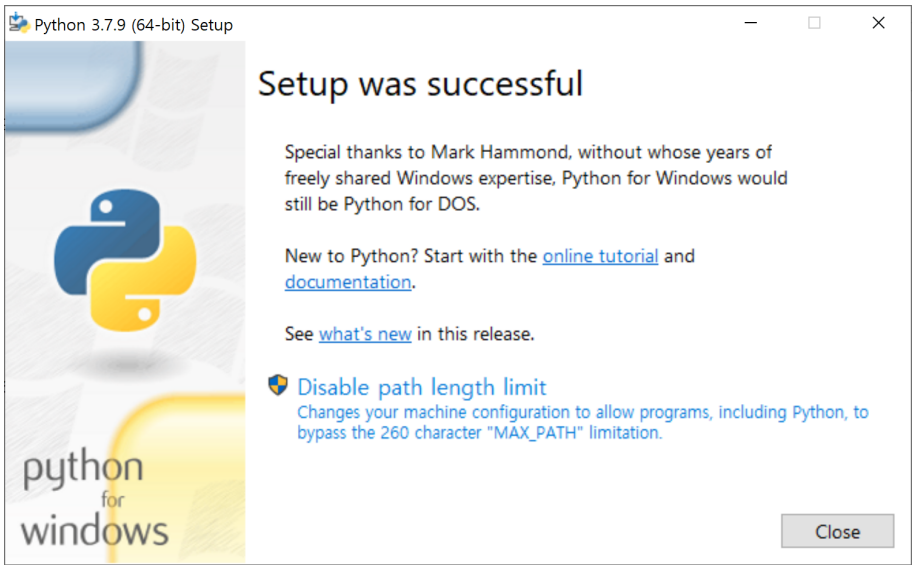


Add Python 3.7 to PATH를 체크하고 Install Now 버튼을 눌러 설치합니다.

> PATH 란?
>
> PATH는 실행 프로그램이 위치한 디렉터리의 집합입니다.
>
> PATH에 실행 프로그램 위치를 지정하면 CLI 환경에서 실행 프로그램의 디렉터리를 지정하지 않아도 자동으로 찾아서 실행합니다.

- **설치 완료**

다음과 같은 화면이 보이면 설치가 완료된 것입니다.



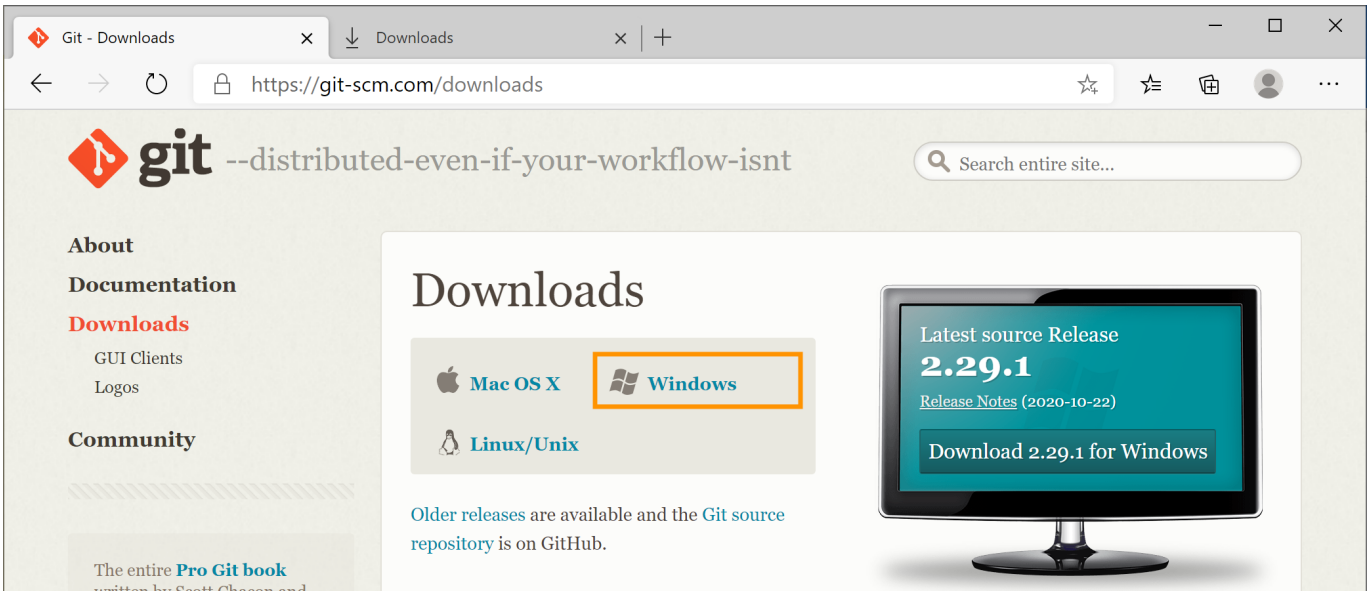
필요에 따라 Disable path length limit을 실행합니다.

Windows는 기본적으로 path length가 제한이 되어 있는데, path의 갯수가 많아지거나 경로가 긴 경우에는 제대로 실행되지 않을 수 있습니다.

- Git 설치 : https://git-scm.com/downloads/

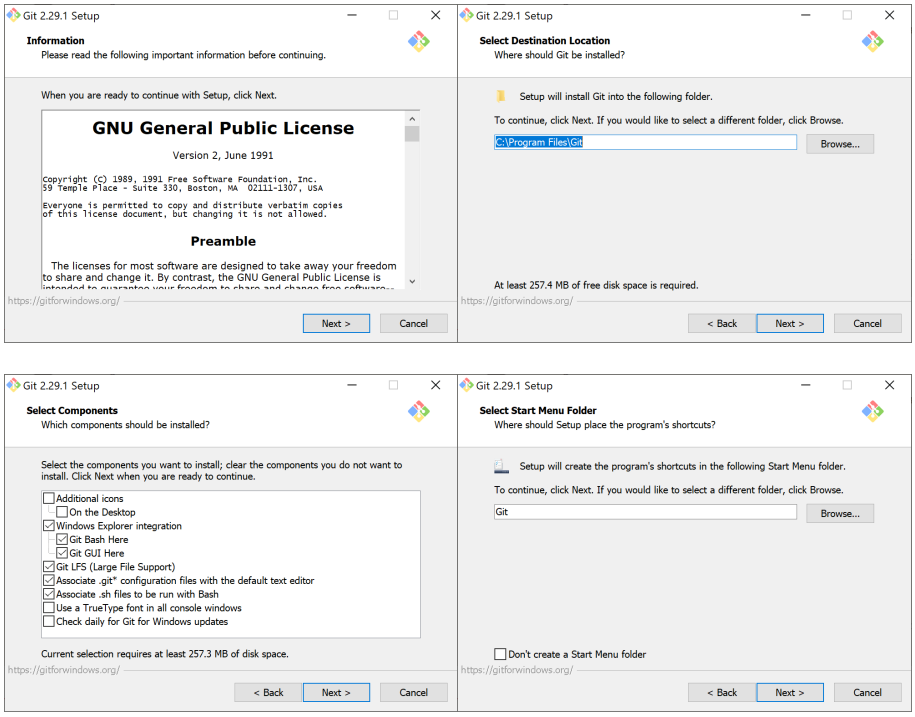
- **다운로드 페이지 접속**

  다운로드 페이지 상단에 있는 다운로드 링크를 이용하여 최신 버전을 다운로드합니다.

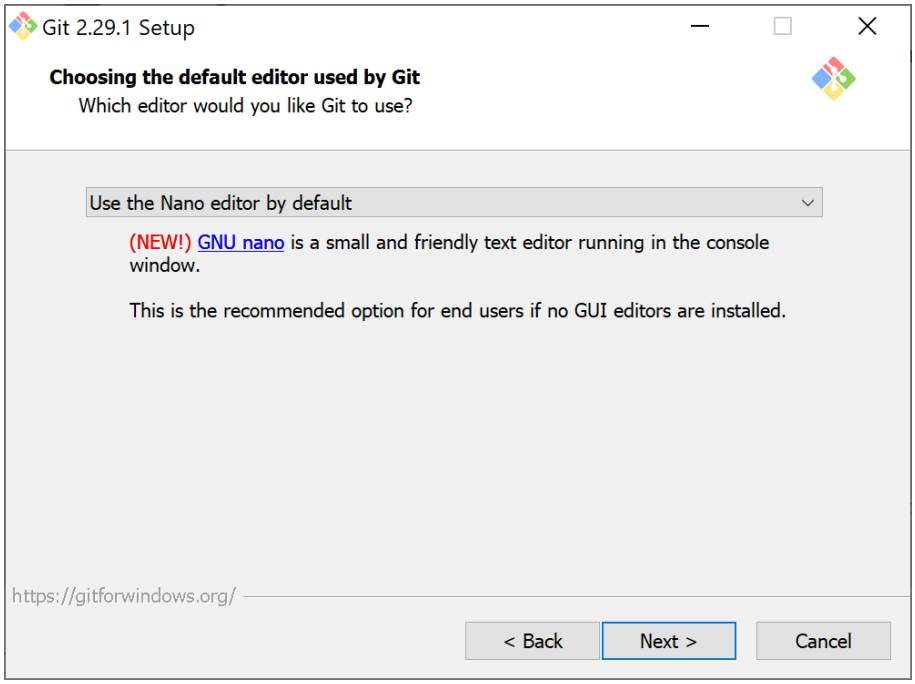


- **Git 설치 프로그램 실행**

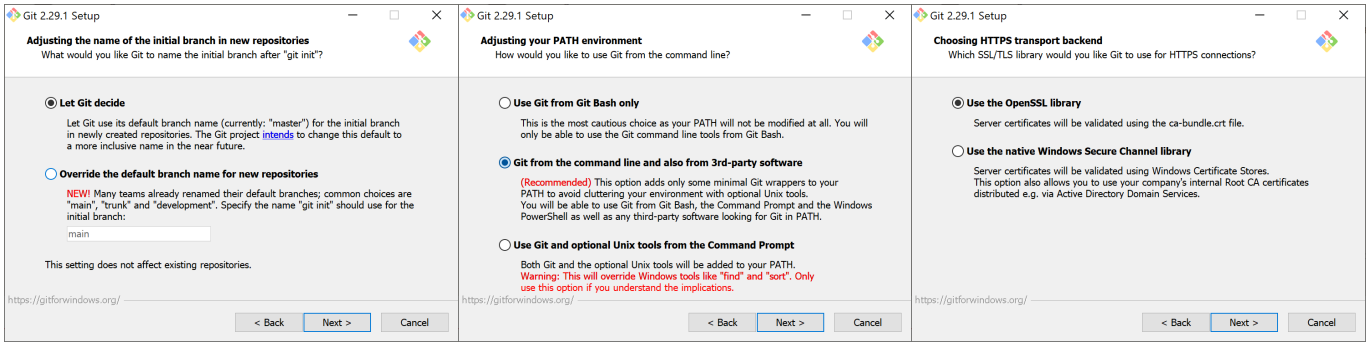
  설치 프로그램에서 Git 구성과 관련하여 10가지 이상의 옵션을 확인합니다.

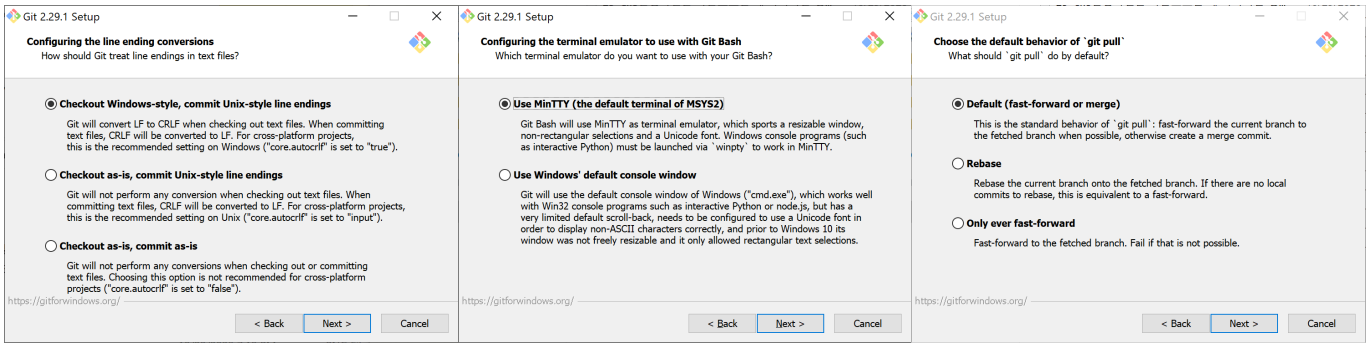


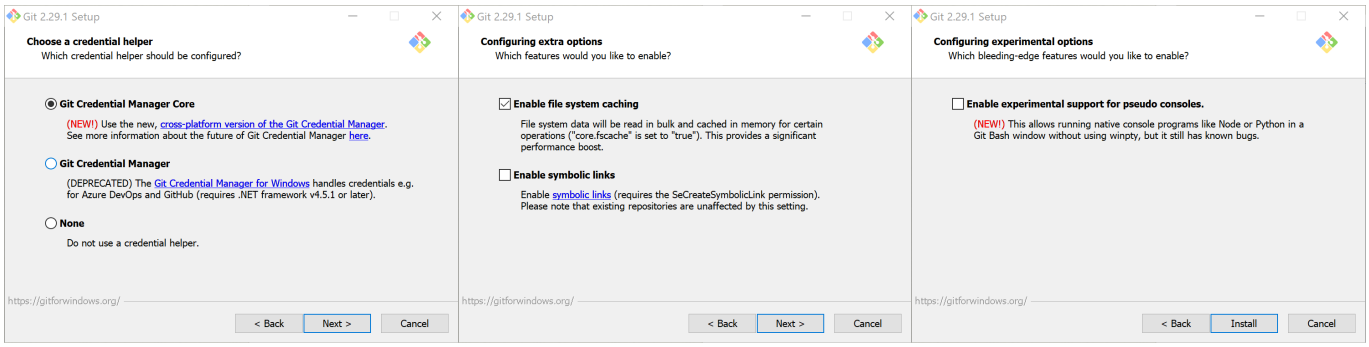
이 빠른시작 가이드에서는 기본 에디터 설정을 제외하고 모두 기본값을 사용합니다.



에디터는 사용하기 편리한 GUI 에디터를 사용하고, GUI 에디터가 없을 경우 VIM 에디터가 아닌 Nano 에디터를 사용하는 것이 좋습니다.

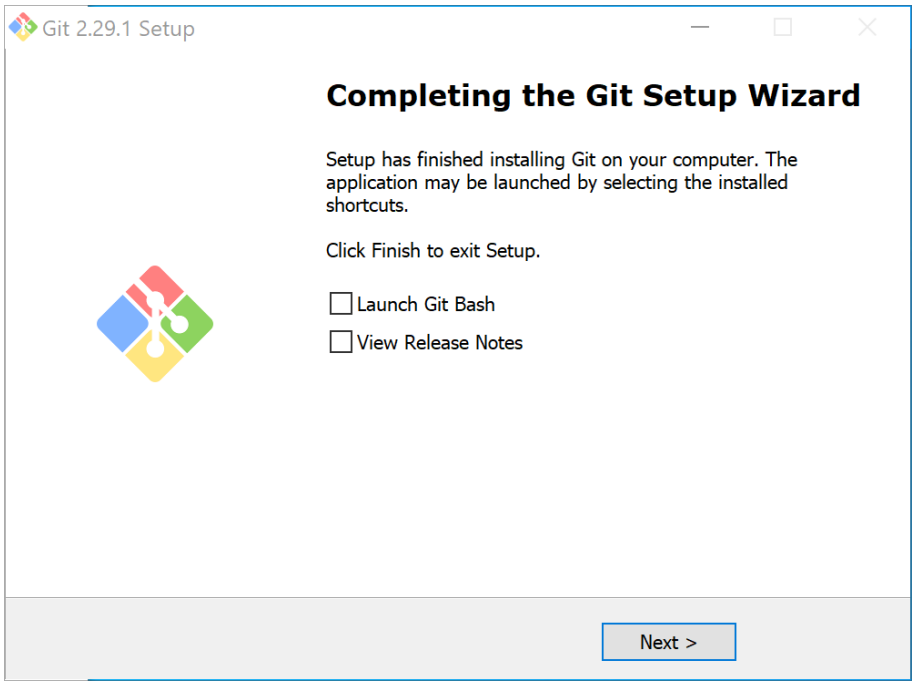






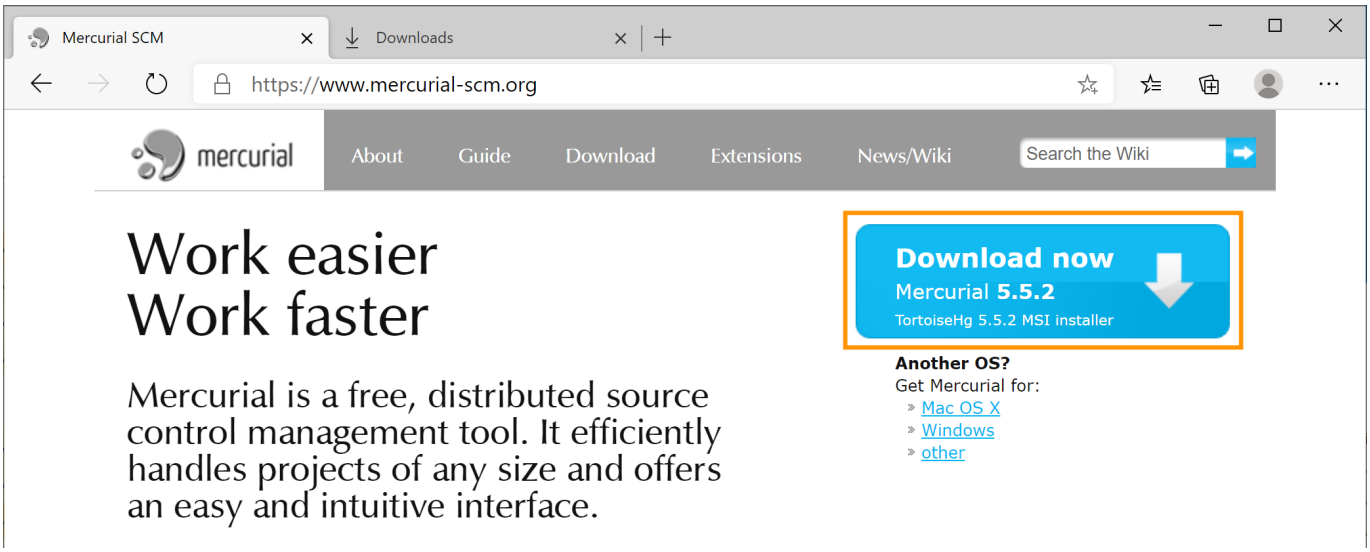
위 옵션들을 기본값으로 설정한 후 설치 버튼을 클릭하여 설치합니다.

- **설치 완료**


Git 2.29.1 Setup
Completing the Git Setup Wizard
Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch Git Bash
View Release Notes
Next >

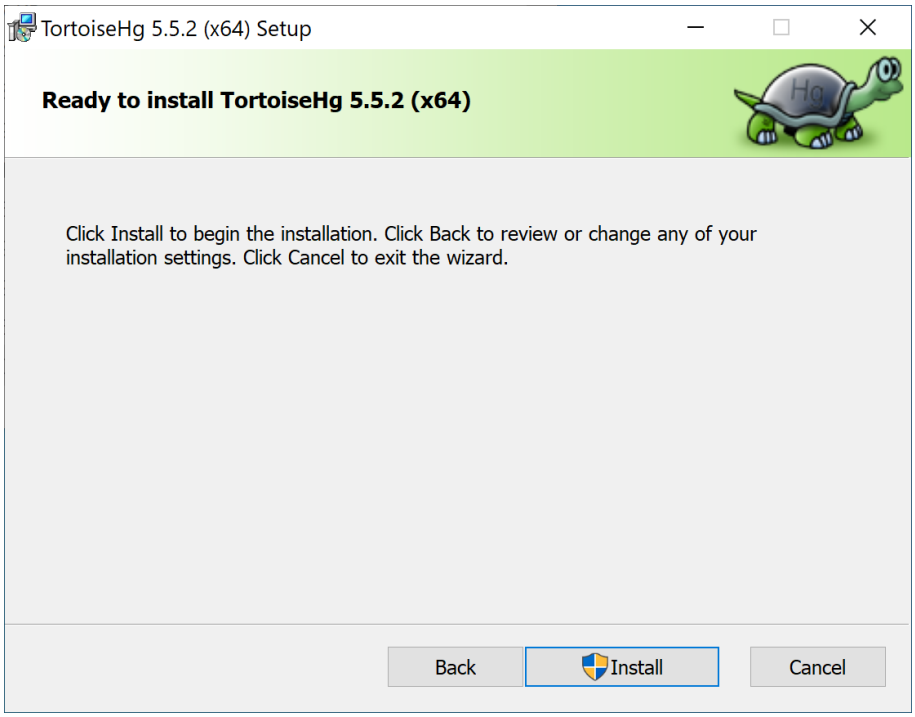
- Mercurial 설치: https://www.mercurial-scm.org/

  - 다운로드 페이지 접속



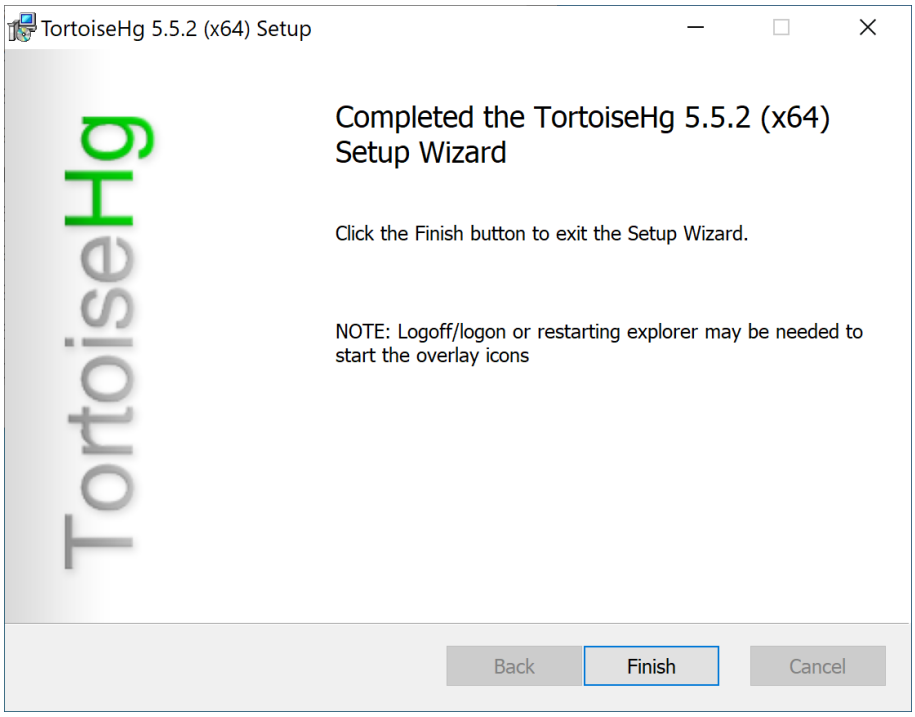
다운로드 페이지 상단에 있는 다운로드 링크를 이용하여 최신버전을 다운로드합니다.

  - Mercurial 설치 프로그램 실행



기본값을 사용하여 설치합니다.

- **설치 완료**



2.2.2.설치 확인

– 명령 프롬프트를 실행한 후 각 명령을 입력해봅니다.

- Python

```
$ python --version
Python 3.7.9
```

3.7.X 버전이 아니거나 버전이 보이지 않으면 `python3 --version` 명령을 입력합니다.

- Git

```
$ git --version
git version 2.29.1.windows.1
```

- Mercurial

```
$ hg --version
Mercurial Distributed SCM (version 5.5.2)
(see https://mercurial-scm.org for more information)
```

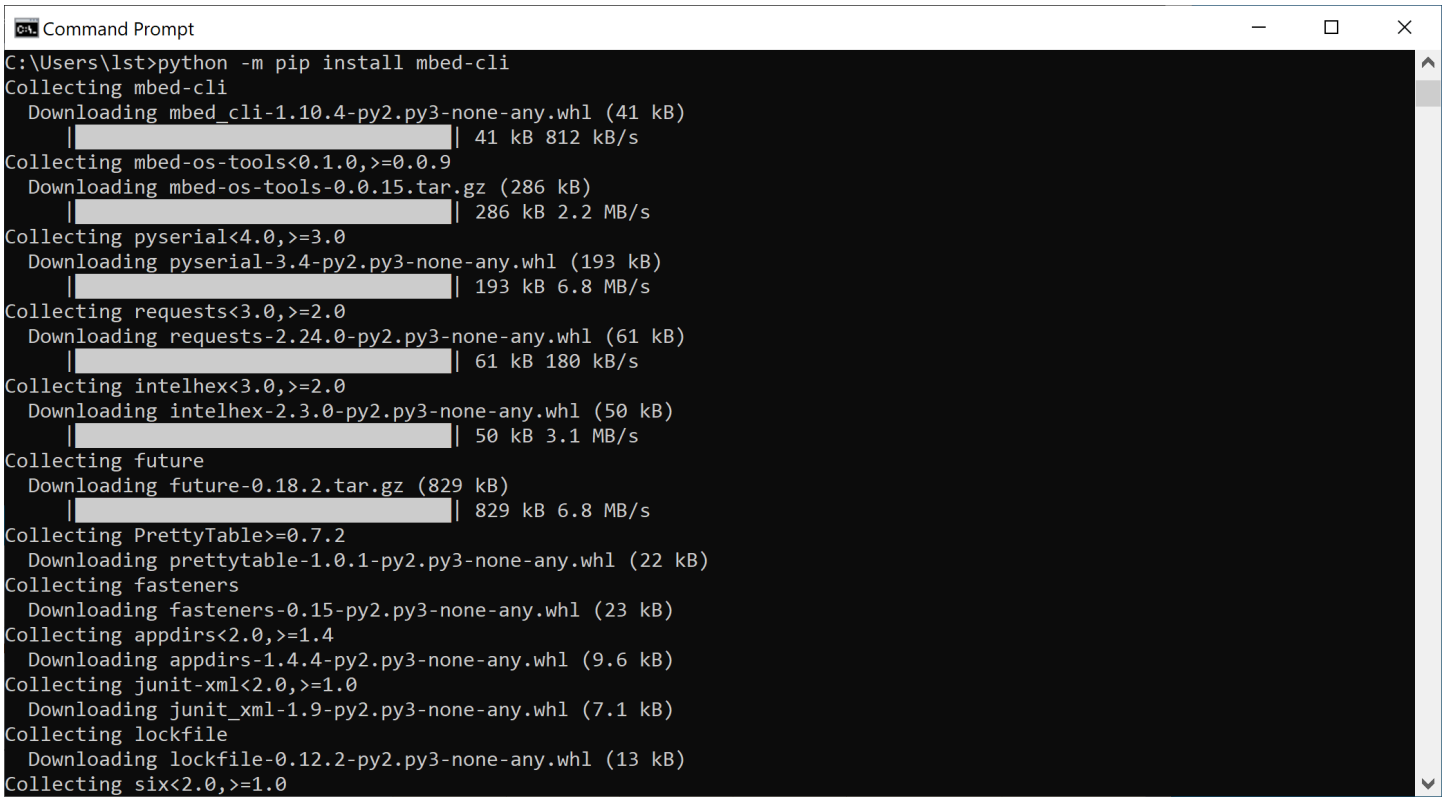
결과가 다르게 나타난다면 명령 프롬프트를 다시 실행하거나 컴퓨터를 다시시작 합니다.

여전히 결과가 다르게 나타난다면 프로그램을 다시 설치합니다.

2.2.3.Mbed CLI 설치

python의 패키지 매니저를 통해 mbed-cli 패키지를 다운로드합니다.

```
$ python3 -m pip install mbed-cli
```



위와 같이 패키지 다운로드가 진행됩니다.

mbed 명령을 사용하여 설치를 확인합니다.

```
$ mbed --version
1.10.4
```

PATH 추가

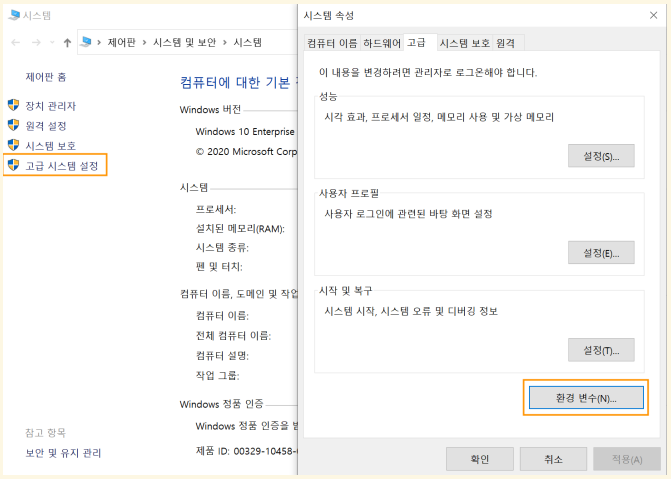
터미널이나 명령 프롬프트에서 mbed 명령을 사용하기 위해 PATH를 추가합니다.

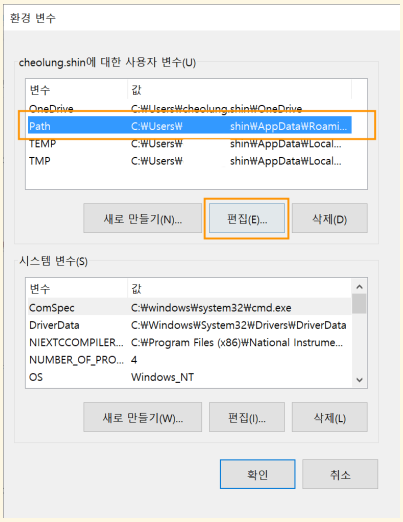
```
Installing collected packages: mbed-cli
  WARNING: The scripts mbed-cli.exe and mbed.exe are installed in 'C:₩Users
₩cheolung.shin₩AppData₩Roaming₩Python₩Python37₩Scripts' which is not on PAT
H.
  Consider adding this directory to PATH or, if you prefer to suppress this
 warning, use --no-warn-script-location.
Successfully installed mbed-cli-1.10.4
```

위와 같은 WARNING이 보인다면 PATH를 추가해야 합니다.

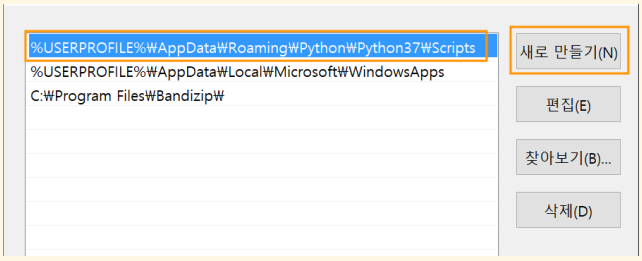
Windows

Win + Pause/Break 단축키 입력





고급 시스템 설정 → 환경 변수 → Path 선택 → 편집



새로 만들기를 클릭하고 아래와 같이 입력

%USERPROFILE%\AppData\Roaming\Python\Python37\Scripts

Linux, macOS

쉘을 실행할 때 실행되는 스크립트에 PATH 정보를 추가합니다.

$ nano ~/.bashrc (shell에 따라 zshrc 등 사용)

$ export PATH=/usr/local/bin/:$PATH

2.2.4.Toolchain 설치

빌드에 필요한 컴파일러, 링커 등 여러가지 도구들이 포함된 패키지를 설치합니다.

Mbed Studio에서는 유료 툴체인인 ARM Compiler 6를 제공하지만, Mbed-CLI는 ARM Compiler 6 를 제공하지 않습니다.

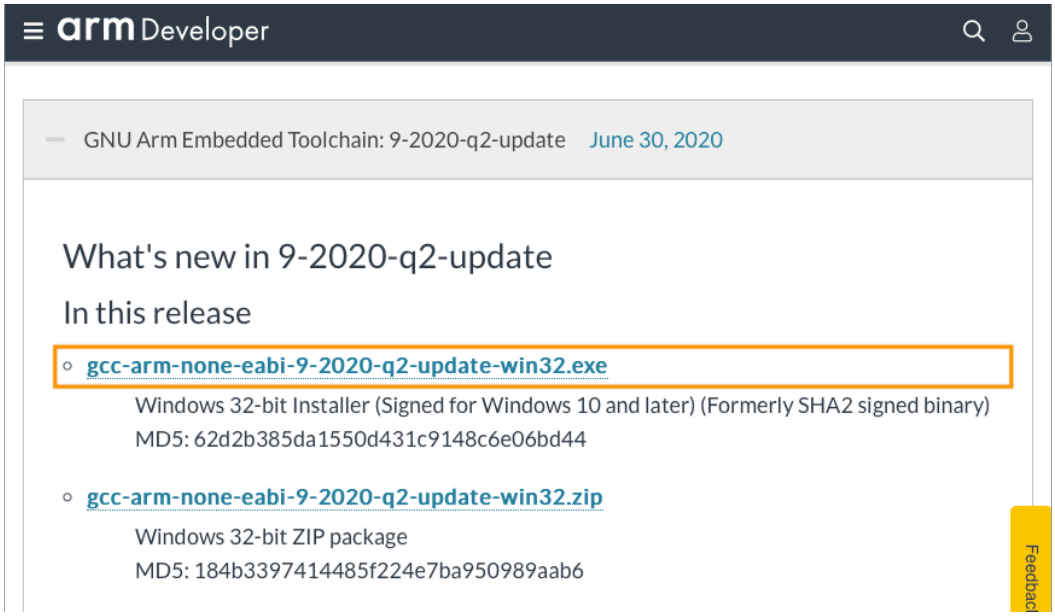
따라서 GNU GPL 라이센스에 의해 무료로 사용 가능한 GNU Toolchain을 사용합니다.

- 설치파일 다운로드

다운로드 사이트에 접속합니다.

https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm/downloads

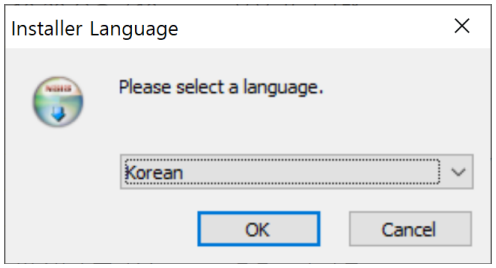
현재(2020년 11월 기준) mbed-cli는 gcc-arm-9버전까지 지원합니다.



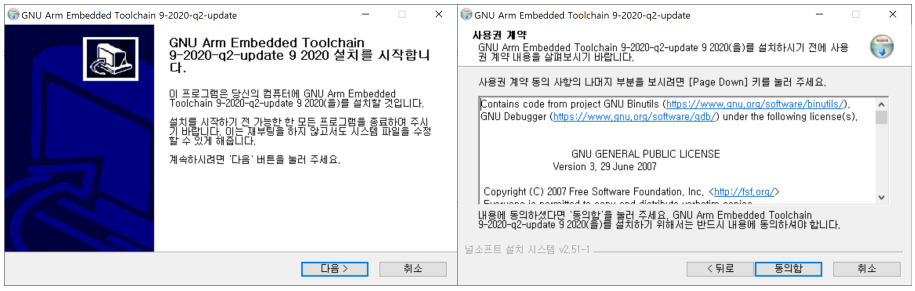
gcc-arm-none-eabi-9-2020-q2-update-win32.exe를 다운로드 합니다.

- 설치 프로그램 실행

설치 프로그램은 프로그램 사용권 동의, 설치 위치 선택 후 설치를 수행합니다.

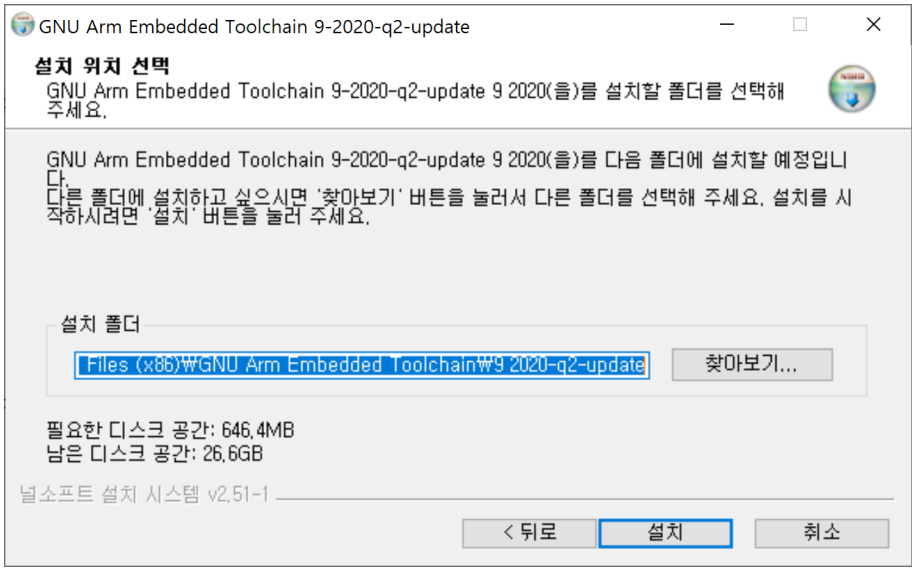


설치 프로그램을 실행하고 설치 프로그램의 언어를 선택합니다.

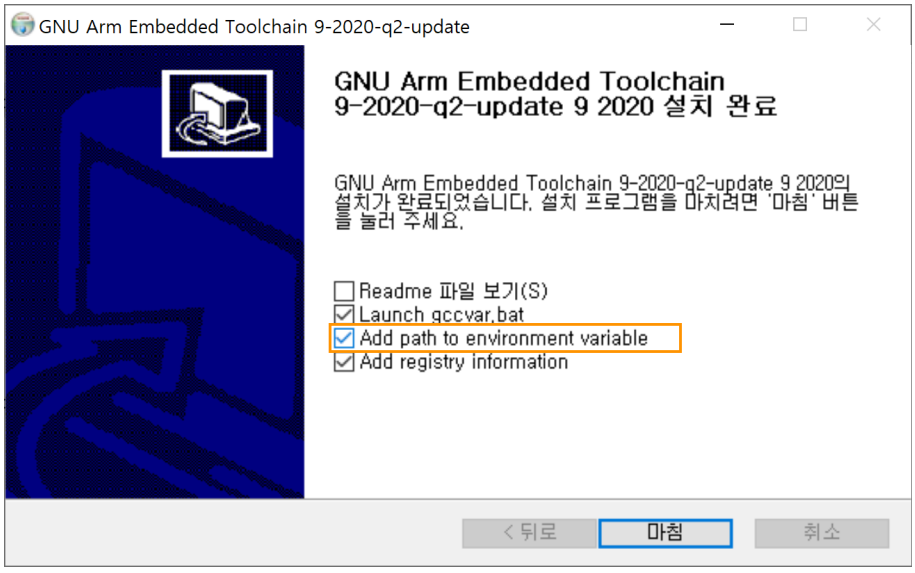


설치 안내 화면에서 '다음'을 클릭하면 프로그램 사용권 계약 화면이 나타납니다.

사용권에 동의한다면 '동의함'버튼을 클릭하여 다음 단계로 진행합니다.



설치 위치를 선택합니다. 기본값으로 두고 '설치' 버튼을 눌러 설치를 시작합니다.



설치가 완료되었습니다.

'Add path to environment variable'을 선택하여 PATH 환경변수를 추가합니다.

2.2.4.1. Mbed 환경 변수 설정

- Mbed CLI 환경 변수 설정

Toolchain의 위치를 Mbed CLI 환경 변수로 저장합니다.

```
$ mbed config –G GCC_ARM_PATH “{설치 위치}”
```

Toolchain 기본 위치 : `C:\Program Files (x86)\GNU Arm Embedded Toolchain\9 2020-q2-update\arm-none-eabi\bin\`

이제 Mbed CLI를 사용하기 위한 환경 구성이 끝났습니다.

3. 빌드 및 배포

Mbed Studio와 Mbed CLI 환경에서 예제 프로그램을 다운로드하고 빌드합니다.

- **Mbed Studio**
- **Mbed CLI**

3.1. Mbed Studio

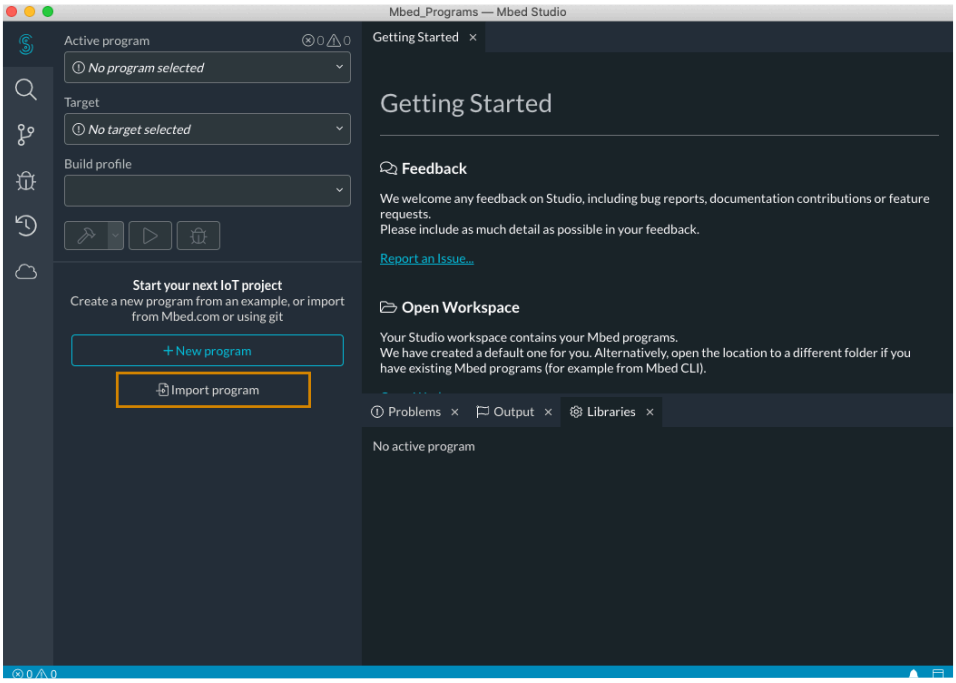
3.1.1. 프로그램 가져오기

우선 Mbed Studio를 통해 예제 프로그램을 다운로드 하는 방법을 알아봅니다.

BioProcessor를 시작하는 가장 쉽고 빠른 방법입니다.

한번의 불러오기 버튼으로 예제 소스와 함께 BioProcessor 라이브러리, mbed os 까지 한번에 다운로드 됩니다.

- Mbed Studio 실행



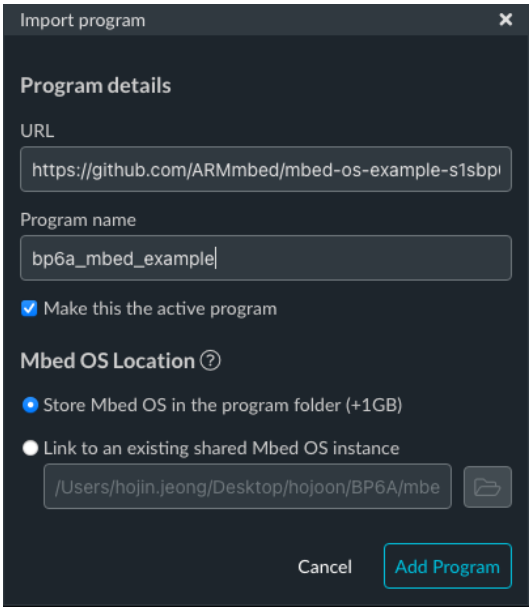
Mbed Studio를 처음 실행하고 로그인하면 위와 같은 화면을 볼 수 있습니다.

Import program 버튼을 클릭하여 예제 프로그램을 가져옵니다.

- 프로그램 가져오기

예제 프로그램을 Github에서 다운로드합니다.

Import program 창에서 git repository 주소만 입력하면 소스코드 다운로드가 시작됩니다.

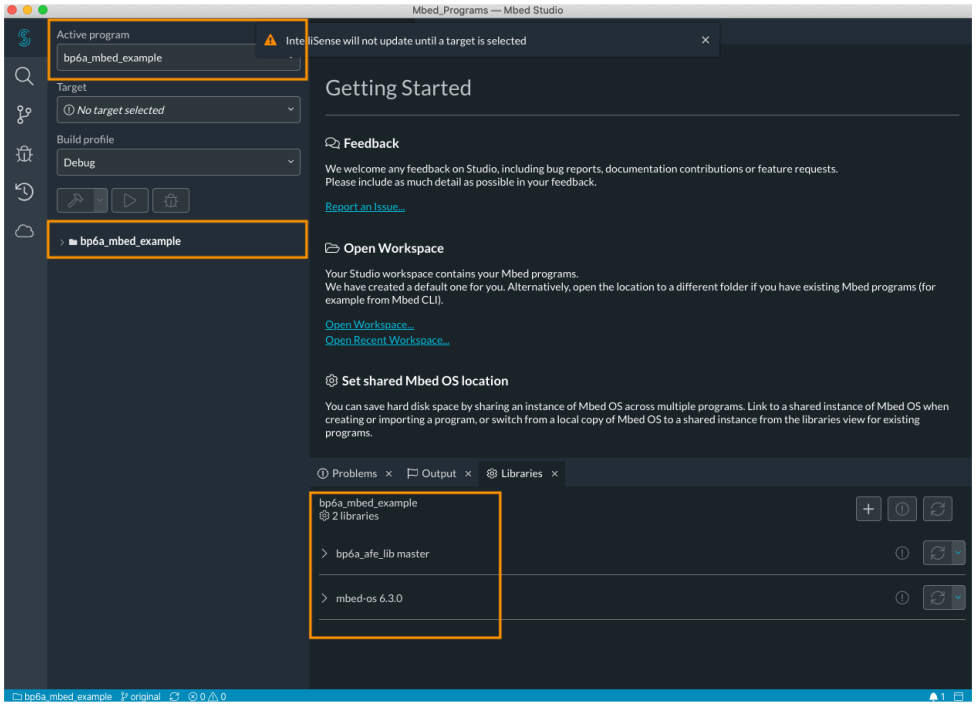


URL : https://github.com/ARMmbed/mbed-os-example-s1sbp6a.git

Program name : bp6a_mbed_example

Mbed OS Location : Store Mbed OS in the program folder

- Link to an existing shared Mbed OS instance 기능은 여러개의 프로그램을 만들 때 Mbed OS 소스코드를 중복해서 다운로드 하는 것을 방지합니다.



성공적으로 불러오면 Active program이 바뀌고 Libraries에 mbed-os와 몇 개 항목이 추가됩니다.

3.1.2. 빌드 및 배포

프로그램을 빌드하기 위해서는 타겟을 설정해야 합니다.

Mbed OS는 여러 종류의 MCU를 지원하도록 설계되어 있습니다. MCU에 맞는 mbed 프로그램을 빌드 하려면 타겟을 설정해야 합니다.

Mbed Studio는 빌드 한 프로그램을 자동으로 BioProcessor에 배포하여 실행하는 기능을 지원합니다.

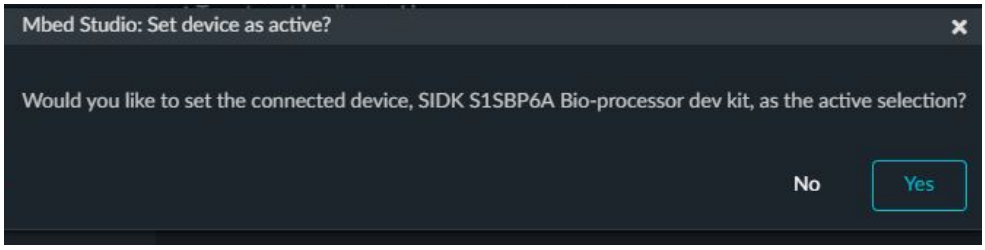
BioProcessor의 MCU는 S1SBP6A입니다.

– 타겟 설정

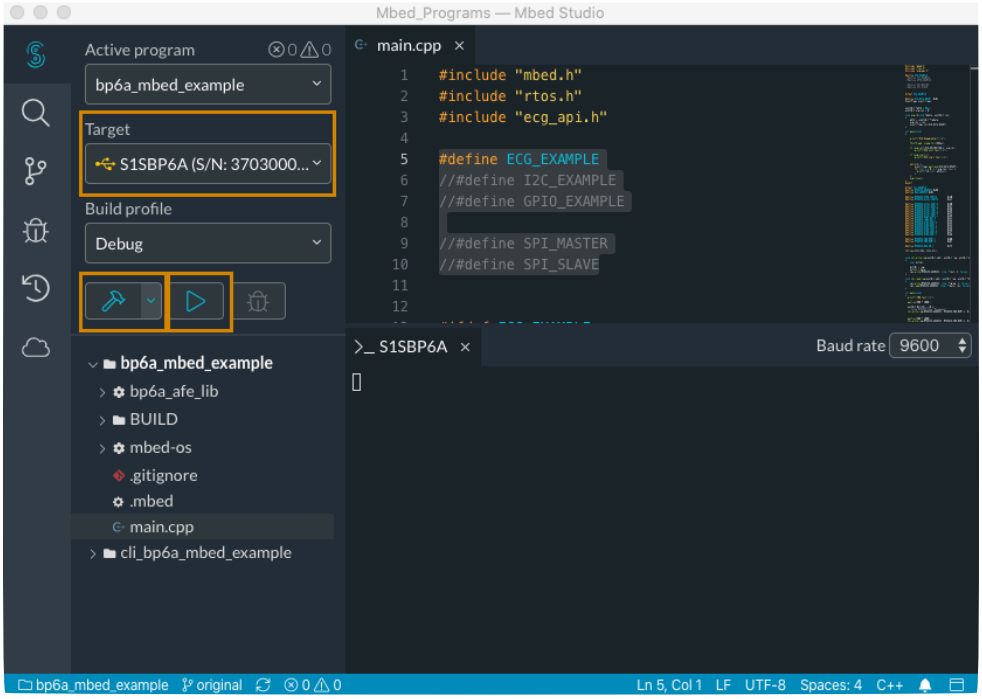
빌드 시의 대상 MCU를 선택합니다.

BioProcessor를 컴퓨터에 연결하면 Mbed Studio가 타겟을 인식하여 설정합니다.

BioProcessor를 컴퓨터에 연결합니다.



연결된 장치를 선택할지 확인하는 창이 나타나면 Yes를 클릭합니다.



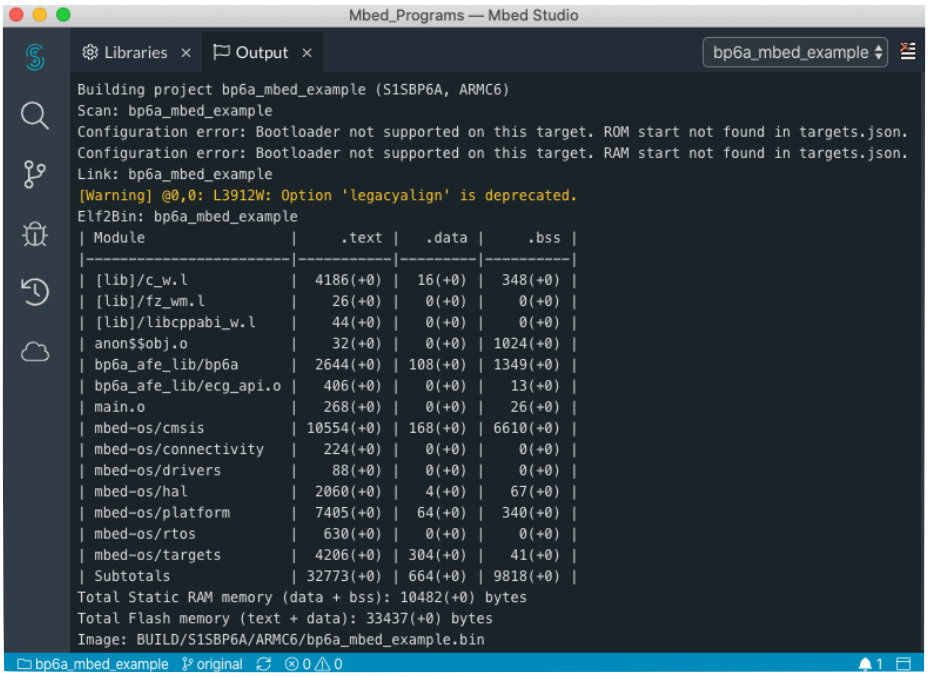
BioProcessor를 인식하면 Target에 USB 아이콘이 생기면서 S1SBP6A로 변경됩니다.

– 빌드

프로그램을 빌드합니다.



Explorer view에서 왼쪽의 빌드 버튼을 클릭하면 빌드를 시작합니다.



예제 프로그램을 수정하지 않고 빌드하면 메모리 및 데이터 크기가 위와 같습니다.

– 실행

빌드 한 프로그램을 컴퓨터에 연결된 BioProcessor에 설치하고 실행합니다.



빌드 버튼 오른쪽에 있는 실행 버튼을 클릭합니다.

Mbed Studio가 프로그램을 빌드 한 후 BioProcessor에 자동으로 설치합니다.

빌드 버튼을 클릭하여 빌드하지 않더라도 실행 버튼을 클릭하면 빌드 후에 설치합니다.

3.2. Mbed CLI

3.2.1. 프로그램 가져오기

Mbed CLI의 Import 명령으로 예제 프로그램을 가져옵니다.

프로그램 이름으로 디렉토리가 자동으로 생성됩니다.

프로그램 이름의 기본값은 Repository의 이름인 bp6a_mbed_example입니다.

```
$ mbed import https://github.com/ARMmbed/mbed-os-example-s1sbp6a.git
```

프로그램 이름을 인수로 추가하면 mbed 프로그램 이름이 바뀝니다.

```
$ mbed import https://github.com/ARMmbed/mbed-os-example-s1sbp6a.git test
```

위 명령을 사용하면 프로그램 이름이 test가 되고 test 디렉토리가 생성됩니다.

3.2.2. 빌드 및 배포

– 타겟 설정

Mbed CLI에서도 Mbed Studio와 같이 자동으로 타겟을 설정하도록 할 수 있습니다.

BioProcessor의 MCU는 S1SBP6A입니다.

- **타겟 감지**

BioProcessor를 컴퓨터에 연결한 후 다음 명령을 입력합니다.

```
$ mbed detect
```

```
↳ mbed detect
[[mbed] Working path "~/BP6A/Mbed_Programs_Example/mbed"

[mbed] Detected S1SBP6A, port /dev/tty.usbmodem143102, mounted /Volumes/DAPLINK, interface version 0254:
[mbed] Supported toolchains for S1SBP6A
| Target  | mbed OS 2 | mbed OS 5 |     ARM     | uARM |  GCC_ARM  | IAR |
|---------|-----------|-----------|-------------|------|-----------|-----|
| S1SBP6A |     -     | Supported | Supported |  -   | Supported |  -  |
Supported targets: 1
Supported toolchains: 2
```

명령을 입력하면 위와 같이 감지한 타겟이 표시됩니다.

S1SBP6A는 ARM과 GCC_ARM 툴체인을 지원합니다.

- **타겟 설정 저장**

  타겟 설정을 저장하기 위해 아래 명령을 입력합니다.

```
$ mbed target S1SBP6A
```

  타겟을 저장하지 않으면 빌드 할 때마다 타겟을 지정해야 합니다.

  빌드 할 때 -m S1SBP6A 인수(Argument)를 추가하여 MCU 타겟을 지정할 수 있습니다.

- **툴체인 설정 저장**

  mbed 프로그램을 빌드 할 때 Mbed-CLI에서 GCC_ARM 툴체인을 사용하도록 지정해야 합니다.

  툴체인을 지정하기 위해 아래 명령을 입력합니다.

```
$ mbed toolchain GCC_ARM
```

  툴체인을 저장하지 않으면 빌드 할 때마다 툴체인을 지정해야 합니다.

  빌드 할 때 -t GCC_ARM 인수를 추가하여 툴체인을 지정할 수 있습니다.

- 빌드

  프로그램을 빌드합니다.

```
$ mbed compile
```

  만약 타겟이나 툴체인을 지정하지 않았다면 아래처럼 필요 인수를 추가해서 실행합니다.

```
$ mbed compile -m S1SBP6A -t GCC_ARM
```

```
Link: bp6a_mbed_example
Elf2Bin: bp6a_mbed_example
| Module                  |     .text |     .data |     .bss |
|-------------------------|-----------|-----------|----------|
| [fill]                  |    68(+0) |     4(+0) |  830(+0) |
| [lib]/c.a               |  7428(+0) | 2108(+0) |   58(+0) |
| [lib]/gcc.a             |   772(+0) |    0(+0) |    0(+0) |
| [lib]/misc              |   188(+0) |    4(+0) |   28(+0) |
| bp6a_afe_lib/bp6a       |  2264(+0) |  108(+0) | 1349(+0) |
| bp6a_afe_lib/ecg_api.o |   372(+0) |    0(+0) |   13(+0) |
| main.o                  |   192(+0) |    0(+0) |   26(+0) |
| mbed-os/cmsis           |  7452(+0) |  168(+0) | 5953(+0) |
| mbed-os/drivers         |    70(+0) |    0(+0) |    0(+0) |
| mbed-os/hal             |  1506(+0) |    4(+0) |   67(+0) |
| mbed-os/platform        |  4166(+0) |  260(+0) |  348(+0) |
| mbed-os/rtos            |   164(+0) |    0(+0) |    0(+0) |
| mbed-os/targets         |  3182(+0) |  304(+0) |   44(+0) |
| Subtotals               | 27824(+0) | 2960(+0) | 8716(+0) |
Total Static RAM memory (data + bss): 11676(+0) bytes
Total Flash memory (text + data): 30784(+0) bytes

Image: ./BUILD/S1SBP6A/GCC_ARM/bp6a_mbed_example.bin
```

  예제 프로그램을 수정하지 않고 빌드하면 메모리 및 데이터 크기가 위와 같습니다.

  컴파일러가 다르기 때문에 Mbed Studio와 결과가 다릅니다.

3.2.3. 배포

Mbed CLI를 사용할 경우 빌드된 프로그램을 BioProcessor에 복사하여 설치할 수 있습니다.

프로그램을 Drag & Drop 하거나 복사 명령을 사용하여 설치할 수 있습니다.

```
$ cp ./BUILD/S1SBP6A/GCC_ARM/bp6A_mbed_example.bin /mnt/sdb/
```

복사 명령은 작업 환경에 따라 다를 수 있습니다.

설치가 완료되면 잠시 후에 BioProcessor가 재시작됩니다.

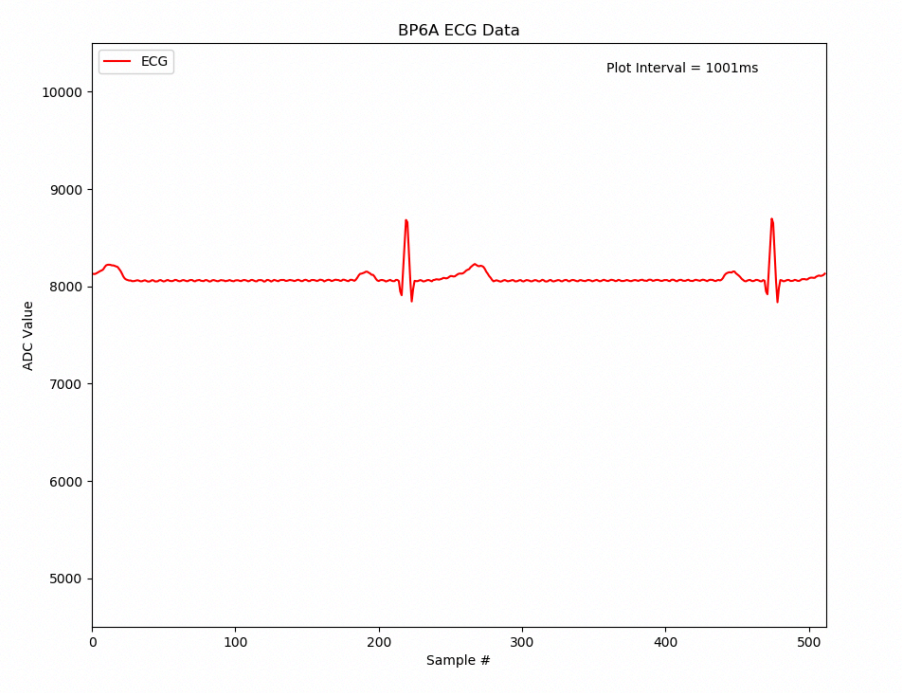
4. 실행, 디버깅

4.1. Python 모듈 설치

```
pip install -r requirements.txt
```

4.2. plot 스크립트 실행

```
python ecg_plot.py
```



5. API Reference

5.1. ECG

5.1.1. ECG 사용하기

ECG API를 사용하는 방법입니다. ECG 데이터를 받기 위해 다음 단계를 따르십시오.

- step 1. ecg_init 함수를 호출하여 ecg 기능을 활성화합니다.
- step 2. ecg_set_decimation_rate 함수를 호출하여 샘플율를 설정합니다.
- step 3. ecg_set_callback 함수를 호출하여 ECG 데이터가 준비되면 알림을 받습니다.
- step 4. ecg_start 함수를 호출하여 ECG 데이터를 받습니다.
- step 5. ECG 데이터가 준비되면 callback 함수가 호출됩니다.

5.1.2. BP6A ECG API

Typedefs

typedef void(*	ecg_callback_t) (void *pdata, uint32_t len)
typedef enum _ecg_decimation_rate_t	ecg_decimation_rate_t

Enumerations

enum	_ecg_decimation_rate_t {
	ECG_NO_DECIMATION = 0,
	ECG_DECIMATION_2 = 1,
	ECG_DECIMATION_4 = 2,
	ECG_DECIMATION_8 = 3,
	ECG_DECIMATION_16 = 4,
	ECG_DECIMATION_INVALID
	}

Functions

int	ecg_set_callback (ecg_callback_t cb)
int	ecg_set_decimation_rate (ecg_decimation_rate_t rate)
int	ecg_init (void)
int	ecg_start (void)
int	ecg_stop (void)
int	ecg_close (void)

5.1.3. BP6A ECG API Documentation

- Typedef Documentation

ecg_callback_t

typedef void(* ecg_callback_t) (void *pdata, uint32_t len)

ECG callback function.

ECG 데이터의 DMA 버퍼가 준비되면 이 함수가 호출됩니다.

ecg_decimation_rate_t

typedef enum _ecg_decimation_rate_t ecg_decimation_rate_t

ECG 샘플링 데이터 다운샘플링 비율.

- Enumeration Type Documentation

_ecg_decimation_rate_t

enum _ecg_decimation_rate_t

ECG 샘플링 데이터 다운샘플링 비율.

- Function Documentation

ecg_set_callback()

int ecg_set_callback (ecg_callback_t cb)

ECG callback function 설정

Parameters

[in] cb 데이터가 준비되면 호출 될 callback fucntion

Return Values

0 : 성공, 기타 : 오류

ecg_set_decimation_rate()

int ecg_set_decimation_rate (ecg_decimation_rate_t rate)

ECG 다운샘플링 비율 설정

Parameters

[in] rate ECG 데이터 다운샘플링 비율(기본 샘플링율 1KHz)

Return Values

0 : 성공, 기타 : 오류

ecg_init()

int ecg_init (void)

설정된 값으로 ECG 기능 초기화

Return Values

0 : 성공, 기타 : 오류

ecg_start()

int ecg_start (void)

ECG 신호 측정 시작

Return Values

0 : 성공, 기타 : 오류

ecg_stop()

int ecg_stop (void)

ECG 신호 측정 종료

Return Values

0 : 성공, 기타 : 오류

ecg_close()

int ecg_close (void)

ECG 초기화 해제 (메모리 확보)

Return Values

0 : 성공, 기타 : 오류

5.2. PPG

5.2.1. Using PPG on BP6A

PPG API를 사용하는 방법입니다. PPG 데이터를 받기 위해 다음 단계를 따르십시오.

- step 1. ppg_init 함수를 prt 시간, 사용할 채널 수와 함께 호출하여 ppg 기능을 활성화합니다.
- step 2. ppg_conf_ch 함수를 호출하여 각 채널에 time slot을 활성화합니다.
- step 3. ppg_set_led_drv 함수를 호출하여 LED가 켜져있는 시간과 전류를 LED 드라이버에 설정합니다.
- step 4. ppg_set_cb 함수를 호출하여 PPG 데이터가 준비되었을 때 callback을 설정합니다.
- step 5. ppg_start 함수를 호출하여 PPG 데이터를 받습니다.
- step 6. PPG 데이터가 준비되면 callback 함수가 호출됩니다.
- step 7. ppg_get_data 함수를 호출하여 특정 time slot의 데이터를 받을 수 있습니다.

5.2.2. BP6A PPG API

Typedefs

typedef void(*	ppg_callback_t) (void)

Functions

int	ppg_conf_ch (int ch, bool t0, bool t1, bool t2, bool t3)
int	ppg_set_led_drv (uint8_t ontime, uint8_t *drv_cur)
void	ppg_set_cb (ppg_callback_t cb0, ppg_callback_t cb1)
uint32_t *	ppg_get_data (int ch, int time_slot, uint32_t *len
int	ppg_init (uint32_t prt_ms, uint8_t n_ch)
int	ppg_start (void)
int	ppg_stop (void)
int	ppg_close (void)

5.2.3. BP6A PPG API Documentation

- Typedef Documentation

ppg_callback_t

typedef void(* ppg_callback_t) (void)

PPG callback function.
PPG 데이터의 DMA 버퍼가 준비되면 이 함수가 호출됩니다.

- Function Documentation

ppg_conf_ch()

```
int ppg_conf_ch (   int  ch,
                    bool t0,
                    bool t1,
                    bool t2,
                    bool t3
                )
```

지정된 인수로 PPG 채널 설정

Parameters

[in]	ch	PPG 채널 번호
[in]	t0	time slot 0 활성화
[in]	t1	time slot 1 활성화
[in]	t2	time slot 2 활성화
[in]	t3	time slot 3 활성화

Return Values

0 : 성공, 기타 : 오류

---

ppg_set_led_drv()

```
nt ppg_set_led_drv  (   uint8_t   ontime,
                        uint8_t * drv_cur
                    )
```

지정된 인수로 PPG LED 드라이버 설정

Parameters

[in]	ontime	LED가 켜져 있는 시간(ms) (최대 : prt / 4)
[in]	drv_cur	4개의 LED 드라이버 전류의 값을 저장한 배열의 포인터 (0~100mA)

Return Values

0 : 성공, 기타 : 오류

---

ppg_set_cb()

```
void ppg_set_cb   (     ppg_callback_t    cb0,
                        ppg_callback_t    cb1
                  )
```

채널 별로 PPG callback function 설정

Parameters

[in]	cb0	채널 0의 데이터가 준비되면 호출 될 callback function
[in]	cb1	채널 1의 데이터가 준비되면 호출 될 callback function

ppg_get_data()

```
uint32_t* ppg_get_data  (     int     ch,
                              int     time_slot,
                              uint32_t *    len
                         )
```

버퍼에서 PPG 데이터 가져오기

Parameters

[in]	ch	PPG 채널 지정
[in]	time	데이터를 가져올 time slot

Enumerator	
ECG_NO_DECIMATION	초당 1K 샘플
ECG_DECIMATION_2	초당 512 샘플
ECG_DECIMATION_4	초당 256 샘플
ECG_DECIMATION_8	초당 128 샘플
ECG_DECIMATION_16	초당 64 샘플
ECG_DECIMATION_INVALID	유효하지 않은 다운샘플링

Return Values

데이터 버퍼의 포인터

ppg_init()

```
int ppg_init    (     uint32_t    prt_ms,
                      uint8_t     n_ch
                )
```

지정된 인수로 PPG 기능 초기화

Parameters

[in]	prt_ms	PPG 펄스 반복 시간(1ms ~ 10ms)
[in]	n_ch	활성 채널의 수 (1 or 2)

Return Values

0 : 성공, 기타 : 오류

ppg_start()

int ppg_start (void)

PPG 신호 측정 시작

Return Values

0 : 성공, 기타 : 오류

ppg_stop()

int ppg_stop (void)

PPG 신호 측정 종료

Return Values

0 : 성공, 기타 : 오류

ppg_close()

int ppg_close (void)

PPG 초기화 해제(메모리 확보)

Return Values

0 : 성공, 기타 : 오류

5.3. BIA

5.3.1. Using BIA on BP6A

PPG API를 사용하는 방법입니다. PPG 데이터를 받기 위해 다음 단계를 따르십시오.

- step 1. bia_init 함수를 호출하여 BIA 기능을 활성화합니다.
- step 2. bia_conf 함수를 tx frequency와 tx amplitude를 함께 호출합니다.
- step 3. bia_calibration 함수를 호출하여 BIA 센서를 보정합니다.
- step 4. bia_set_callback 함수를 호출하여 BIA 데이터가 준비되었을 때 callback을 설정합니다.
- step 5. bia_start 함수를 호출하여 BIA 데이터를 받습니다.
- step 6. BIA 데이터가 준비되면 callback 함수가 호출됩니다.
- step 7. bia_cal_impedance 함수를 호출하여 신체 임피던스를 받을 수 있습니다.

5.3.2. BP6A BIA API

Typedefs

typedef void(*	bia_callback_t) (void *pdata, uint32_t len)

Functions

int	bia_conf (uint16_t tx_freq, uint8_t tx_amp
int	bia_set_callback (bia_callback_t cb)
void	bia_calibration (void)
uint32_t *	bia_cal_impedance (uint16_t *pdata, uint32_t len)
int	bia_init (void)
int	bia_start (void)
int	bia_stop (void)
int	bia_close (void)

5.3.3. BP6A BIA API Documentation

- Typedef Documentation

bia_callback_t

typedef void(* bia_callback_t) (void *pdata, uint32_t len)

BIA callback function.

BIA 데이터의 DMA 버퍼가 준비되면 이 함수가 호출됩니다.

- Function Documentation

bia_conf()

```
int bia_conf (      uint16_t    tx_freq,
                    uint8_t     tx_amp
              )
```

지정된 인수로 BIA 설정

Parameters

[in]	tx_freq	Tx_frequency 1KHz ~ 1Mhz(unit : KHz)
[in]	tx_amp	Tx amplitude (0~3 : 300mv~600mV)

Return Values

0 : 성공, 기타 : 오류

bia_set_callback()

```
int bia_set_callback    (    bia_callback_t    cb    )
```

BIA callback function 설정

Parameters

[in]	cb	BIA 데이터가 준비되면 호출될 callback function

Return Values

0 : 성공, 기타 : 오류

bia_calibration()

```
void bia_calibration    (    void    )
```

BIA 보정

bia_cal_impedance()

```
uint32_t bia_cal_impedance    (    uint16_t *    pdata,
                                   uint32_t      len
                              )
```

BIA 신체 임피던스 계산

Parameters

[in]	p_data	bia raw 데이터의 포인터
[in]	len	bia data의 길이

Return Values

신체 임피던스

bia_init()

int bia_init (void)

지정된 인수로 BIA 기능 초기화

Return Values

0 : 성공, 기타 : 오류

bia_start()

int bia_start (void)

BIA 신호 측정 시작

Return Values

0 : 성공, 기타 : 오류

bia_stop()

int bia_stop (void)

BIA 신호 측정 종료

Return Values

0 : 성공, 기타 : 오류

bia_close()

int bia_close (void)

BIA 초기화 해제 (메모리 확보)

Return Values

0 : 성공, 기타 : 오류